

Birla Institute of Technology & Science, Pilani
Second Semester 2014-2015, CS F111 Computer Programming
Comprehensive Exam (Open Book) PART A Set X

Suggested time: 60 minutes

16/5/15, 2.00 - 4.00 PM

Max. Marks: 44

NOTE: 1. There are 22 questions in Part A. Each question has exactly one answer correct and carry 2M.

2. There is no negative marking. No overwritten answers will be rechecked.

3. For each question, write the correct option (A, B, C, or D) at the designated place on the back of this sheet.

ID	NAME		
Q1. What is the output assuming that the size of char is 1 byte and we use 2's complement representation. <pre>int main() { char c = 125; c = c + 10; printf("c = %d", c); return 0; }</pre> A) 135 B) -121 C) -7 D) compile time error	Q2. The output of the code below is: <pre>int x=-100; int main() { if (x) printf("hi"); else printf("how are u"); return 0; }</pre> A) hi B) how are you C) Compile time error D) None of the mentioned	Q3. In the following program, how many times scanf() statement will be executed ? <pre>int main() { int t,i; for (t=2;scanf("%d",&i) - t ; printf("%d\n",i)) printf("%d--",t- -); return 0; }</pre> A) 1 B) 2 C) 3 D) The loop runs infinitely.	Q4. Consider an integer array mat[3][3]. What will the following loop do? <pre>q = mat[0][2]; for (i = 1; i < 3; i++) { if (mat[i][2-i] < q) { q = mat[i][2-i]; } }</pre> A) Finds the minimum value in matrix. B) Finds the minimum value in main diagonal, i.e. at index [0][0] [1][1] [2][2] C) Finds the minimum value in secondary diagonal i.e. at index [0][2] [1][1] [2][0] D) Finds the maximum value in matrix.
Q5. The expression 5-2-3*5-2 will give 18 if: A) – is left associative and, * has precedence over – B) – is right associative and, * has precedence over – C) – is right associative and, – has precedence over * D) – is left associative, – has precedence over *	Q6. What is the output of the following program? <pre>int main() { static int i=3; printf("%d",i--); return i>0 ? main():0; }</pre> A) 3210 B) 321 C) 32 D) 3	Q7. What is the output of the following piece of code: <pre>enum day { sun = 1, mon, tue = 5, wed, thu=10, fri, sat}; int main() { printf("%d ", sun+mon-tue+wed-thu+fri-sat); return 0; }</pre> A) -2 B) -7 C) 12 D) 47	Q8. What is the o/p of this program? <pre>int main(){ char arr[10]="some"; while(strcmp("some",arr)) printf("Strings are not equal!! \n"); return 0; }</pre> A) Strings are not equal B)Strings are not equal (Infinite Times) C) No output D) Compilation Error
Q9. What will be the output of the following C program: <pre>int main(void) { int i=4; switch(i) { default:printf("zero"); case 1: printf("one"); break; case 2: printf("two"); break; case 3: printf("three"); break; } return 0; }</pre> A) zero B) Display nothing C) zeroone D) run time error	Q10 What should be the output of this program? <pre>int main(void) { int num[11] = {0, 4, 2, 6, 4, 8, 6, 10, 8, 2, 8}; printf("%d", fun(num+3)); return 0; } int fun(int *n) { int j=1, val=0; while(j++<=3) { val+=(*(n+1)>4?-1); } return val; }</pre> A) 2 B) 4 C) -3 D) 38	Q11 What is the o/p of the following program: <pre>int i=3,a=1,b=1; void fun() { int a=0; static int b=0; a++; b++; } int main() { for(i=0;i<5;i++) { fun(); a++; b++; } printf("a=%d b=%d\n",a,b); }</pre> A) a=0 b=6 B) a=5 b=5 C) a=5 b=6 D) a=6 b=6	Q12 What function from the standard C library is this? <pre>void mysteryFunction(char *x, char *y) { char *marker, *marker2; for(marker = x, marker2=y; *marker2; ++marker2, ++marker) *marker = *marker2; *marker = *marker2; }</pre> A) strcpy B) strcat C) strcmp D) strncpy Q13 If <pre>int i[] = {10, 20, 30, 40}; int *pa[] = {i+1, i, i+3, i+2};</pre> What is the value of *pa[1]? A) 10 B) 20 C) 30 D) 40 Q14 Considering 8-bit 2's complement representation, If X=(EA)₁₆ and Y = (71)₈, perform the operation: (-X-Y). The answer in hexa- decimal number system is: A) 1D B) E3 C) D1 D) 3E

Q15. What is the o/p of this program? <pre>int SIZE=10; void size(int arr[]) { printf("%d",sizeof(arr)); } int main() { int arr[SIZE]; printf("%d ",sizeof(arr)); size(arr); return 0; }</pre> A) 4 40 B) 40 4 C) 40 40 D) 4 4	Q16. What is the output of the following program: <pre>union som_union { int num1; char arr[4]; int num2; }; int main() { union som_union t; t.num1 = 0; t.arr[1] = 'A'; printf("%s", t.arr); return 0; }</pre> A) Compiler error. B) A C) Nothing is printed D) Some garbage character followed by A	Q17. What is the output of the following code: <pre>struct folder{ int id; struct folder subfolder; }; int main() { struct folder f; f.id = 10; printf("%d ", f.id+20); return 0; }</pre> A) 30 B) 10 C) runtime error D) compile time error	Q18. The following recursive function calculates the value of a number (base) raised to a power. Assuming that power is a non-negative integer, what should X and Y be replaced with? <pre>int PR (int base, int power) { int ans; if (power == 0) ans = ____X____; else ans = base * ____Y____; return ans; }</pre> A) X with 0 and Y with PR (base, power-1) B) X with 1 and Y with PR(base-1, power) C) X with 0 and Y with PR(base-1, power) D) X with 1 and Y with PR(base, power-1)
Q19. Consider the following structure. Which of the following statements are syntactically not correct to allocate memory for the variable of type struct ownernode? <pre>struct ownernode { owner own; struct ownernode *next; }; typedef struct ownernode* Onode;</pre> A) Onode temp; temp=(Onode)malloc(sizeof(struct ownernode)); B) struct ownernode *temp; temp= (struct ownernode *)malloc(sizeof(struct ownernode)); C) struct ownernode *temp; temp= (Onode)malloc(sizeof(struct ownernode)); D) struct ownernode *temp; temp= (ownernode *)malloc(sizeof(struct ownernode));	Q20. What is the output of the following code: <pre>int main() { struct node{ int m; struct node *next; } x, y, z, *p; x.m = 10; y.m = 20; z.m = 30; x.next = &y; y.next = &z; z.next = NULL; p = x.next; while(p != NULL){ printf("%d ", p->m); p = p->next; } return 0; }</pre> A) 20 30 B) 10 20 C) 10 20 30 D)Runtime error	Q21. To find whether two bit patterns are identical or not, perform operation X on two numbers and check the result for all zeroes. What is this operation X ? A) AND B) NAND C) XOR D) XNOR	Q22. Convert the following binary number num in hexadecimal form. Assume 16-bit signed magnitude representation here. num = 110100110 A) FFA6 B) 80A6 C) FOA6 D)8FA6

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
 COMPREHENSIVE EXAMINATION, SECOND SEMESTER 2014-15,
 CS F111 COMPUTER PROGRAMMING, **PART A Set X ANSWER SHEET (OPEN BOOK)**

ID		NAME	
-----------	--	-------------	--

Write your answers legibly in the space provided below.

Q. No.	1	2	3	4	5	6	7	8	9	10	11
Correct Option	B	A	B	C	C	B	B	C	C	C	D
Q. No.	12	13	14	15	16	17	18	19	20	21	22
Correct Option	A	A	A	B	C	D	D	D	A	C	B

Total Correct Answers:

x 2 =

Total Marks

Birla Institute of Technology & Science, Pilani
Second Semester 2014-2015, CS F111 Computer Programming
Comprehensive Exam (Open Book) PART B

Date: May 16, 2015

Time: 2.00 - 4.00 PM

Max. Marks: 45

NOTE: Each question is partially implemented. Write the missing parts as your answers at the designated place in the separate answer sheet provided. Write only one statement in each box.

Q1. Consider a two dimensional character array named as **Arr[row][col]**.

[1+2+2+1+2+2+2]

A sequence **S1** in the i^{th} row of the array is defined as a combination of two characters (**Arr[i][j]**, **Arr[i][k]**) where $j < k$. In the example array, **Arr**, shown below (Figure 2) : < ae, ac, ad, ec, ed, cd> are all the possible **S1** sequences of the 0th row.

A sequence **S2** of i^{th} row is defined as a combination of two characters (**Arr[i][j]**, **Arr[i][k]**) where $j < k$ AND $k = j + 1$. In the example array, **Arr**, shown below (Figure 2): <db, ba, ac> are all the possible **S2** sequences of the 1st row.

The function **Find_Sequence()** (Figure 1) takes a 2-D character array **Arr**, number of columns and rows as an input. It finds for each possible **S1** sequences of the 0th row of **Arr**, the number of occurrences of **S2** sequences in the remaining rows of the **Arr**.

Example array Arr[] **Output of Find_Sequence()**

a	e	c	d
d	b	a	c
e	c	a	d
a	d	b	c

ae occurs 0 times.
ac occurs 1 times.
ad occurs 2 times.
ec occurs 1 times.
ed occurs 0 times.
cd occurs 0 times.

Figure 2 [Q1]

```
void Find_Sequence(int col, char Arr[][col], int row) {
    int i,j,k,m, count=0;
    char f_char_tocheck, s_char_tocheck, s_char, f_char;
    for (i=0;i<(col-1);i++)
    {
        for (j=i;j<(col-1);j++)
        {
            //Characters from first row to search
            f_char_tocheck = Arr[0][i];
            s_char_tocheck = Arr[0][j+1];
            for (k=1;k<row;k++)
            {
                for (m=0;m<(col-1);m++)
                {
                    //Characters from remaining rows to compare
                    f_char = Arr[k][m];
                    s_char = Arr[k][m+1];
                    if ((f_char_tocheck == f_char) && (s_char_tocheck == s_char))
                        count++;
                } //innermost for loop
            }
            printf ("%c %c occurs %d times\n",
                    f_char_tocheck, s_char_tocheck, count);
            count=0;
        }
    } //outermost for loop
}
```

Figure 1 [Q1]

Q2. Assume that a particular college for its admission

considers a total of best 4 out of 6 subjects. For this, a candidate specifies the best 4 subjects whose total marks are to be considered. Function **compute_marks()** (Figure 4) is implemented for it.

However, college also validates this by adding the marks of best 4 subjects. This is done based on actual marks of 6 sub's without considering students choice. In figure 3, function **compute_marks_college()** is implemented for it.

To represent each student, a structure named student is defined. Its fields are as follows: (a) ID is a unique ID number of student, (b) array "sub" stores the marks of 6 subjects, where sub[0] is a first subject. Variable "choice" is a six digit integer, where each of its digits is either 0 or 1. It represents the choices of subjects given by student; 0 means don't consider the subject among best 4 and 1 means consider it. The most significant digit represents the first subject and least significant one represents the sixth subject. For example, if choice = 101101, it means 2nd and 5th subject (whose marks are in sub[1] and sub [4]) should not be considered among best 4. Assume marks in a subject can be between 0 and 100, both inclusive. "bfm" stores the marks of best four subjects.

```
float compute_marks_college (student s){
    int i,j,min;
    float sum=0;
    for (i=0;i<2;i++)
    {
        min = 0;
        for (j=1;j<6;j++)
        {
            if (s.marks.sub[j] < s.marks.sub[min])
                min = j;
            s.marks.sub[min] = 999;//hint
        }
        for (i=0;i<6;i++)
        {
            if (s.marks.sub[i] != 999)
                sum += s.marks.sub[i];
        }
        return sum;
    }
}
```

Figure 3 [Q2]

```
typedef struct{
    char ID[12];
    struct{
        float sub[6];
    } marks;
    int choice;
    float bfm;
} student;

void compute_marks (student *s)
{
    float sum=0;
    int bit, pos=5;
    while (pos >= 0) {
        bit = (s->choice) % 10;
        s->choice /= 10;
        switch (bit)
        {
            case 0: break;
            case 1: sum += s->marks.sub[pos];
        }
        pos --;
    }
    s->bfm = sum;
}
```

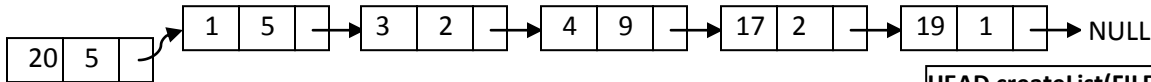
Figure 4 [Q2]

Figure 5 [Q2]

Q3. An integer type *sparse-array* is a one dimensional array whose most of the elements are 0's (zero). In order to save memory required to store such array, the non-zero elements of *sparse-array* can be stored in the form of a singly linked list. The node of a list (Figure 6) contains (a) index of non-zero element, (b) element itself, and (c) pointer to next node of the list. Moreover, the nodes of the list are sorted with respect to index. The head node of the list (Figure 7) contains (a) size of original array, (b) total number of non-zero elements in sparse array (means total number of nodes in linked list), and (c) pointer to first node of the list. For example, consider the following sparse array:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
element	0	5	0	2	9	0	0	0	0	0	0	0	0	0	0	0	0	2	0	1

The linked list representation of the above array is as follows:



Q3 (a) The function, **createList()** (Figure 8), takes a FILE pointer of an already opened file as input argument. Each row of this file contains an integer number. The first two integer entries of the file indicate respectively the size of sparse-array and number of non zero elements (**nze**). It is followed by **nze** number of integer pairs; where first number of this pair represent index of non-zero element in sparse array, and second number represent the non-zero element itself. This function creates a list by reading elements from this file and returns a pointer of its head node. Assume that integer pairs in file are in sorted order wrt to index.

```

HEAD add_merge(HEAD list1,HEAD list2) {
    HEAD newlist;
    NODE temp, i, j, k, c;
    newlist=(HEAD)malloc(sizeof(struct
head));
    if(list1->size > list2->size)
        newlist->size=list1->size;
    else
        newlist->size=list2->size;
    newlist->nze =0;
    newlist->link=NULL;
    i=list1->link;
    j=list2->link;
    k = c =(NODE)malloc(sizeof(struct node));
    while(i!=NULL && j!=NULL)
    {
        if(i->index < j->index)
        {
            k->next=i;
            i=i->next;
            newlist->nze =newlist->nze+1;
            list1->nze = list1->nze-1;
        }
        else if(i->index > j->index)
        {
            k->next=j;
            j=j->next;
            newlist->nze =newlist->nze+1;
            list2->nze = list2->nze-1;
        }
        else if(i->index == j->index)
        {
            temp= (NODE)
                malloc(sizeof(struct node));
            temp->val=i->val + j->val;
            temp->index =i->index;
            k->next=temp;
            i=i->next;
            j=j->next;
            newlist->nze =newlist->nze+1;
            list1->nze = list1->nze-1;
            list2->nze = list2->nze-1;
        }
        k=k->next;
    } //end of while loop
    if(i==NULL)
    {
        k->next=j;
        newlist->nze=
            newlist->nze+list2->nze;
    }
    else
    {
        k->next=i;
        newlist->nze=
            newlist->nze+list1->nze;
    }
    newlist->link=c->next;
    return(newlist);
}

```

Figure 9 [Q3 (b)]

<pre> struct node { int index; int val; struct node *next; }; typedef struct node * NODE; </pre> Figure 6 [Q3]	<pre> struct head { int size; int nze; NODE link; }; typedef struct head * HEAD; </pre> Figure 7 [Q3]
---	--

```

HEAD createList(FILE *fp1) {
    int num1,num2,i,hsize,hval;
    NODE temp,cur;
    HEAD h;
    fscanf(fp1,"%d %d",&hsize,&hval);
    h=(HEAD)malloc(sizeof(struct head));
    h->size=hsize;
    h->nze =hval;
    h->link=NULL;
    for(i=2;i<=h->nze+1;i++)
    {
        fscanf(fp1,"%d %d",&num1,&num2);
        temp=(NODE)malloc(sizeof(struct node));
        temp->index=num1;
        temp->val=num2;
        temp->next=NULL;
        if (h->link==NULL)
            h->link=temp;
        else {
            cur=h->link;
            while(cur->next!=NULL)
                cur=cur->next;
            cur->next=temp;
        }
    }
    return(h);
}

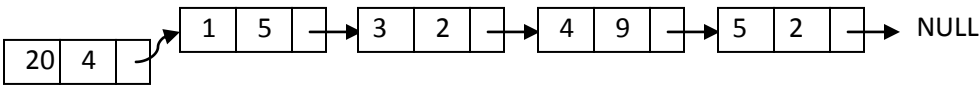
```

Figure 8 [Q3 (a)]

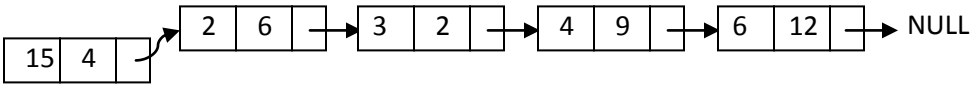
Q3 (b) The function, **add_merge()**, (figure 9), takes two lists as argument. It *add and merge* them into one list and returns head pointer of this newly created list. To add and merge, start traversing the two lists simultaneously. If index stored in both the nodes of the list is same, then insert a new node in the new list with its value **val** set to addition of **val** values of two nodes. Rather, if they are not same, then insert the node of the corresponding list into new list such that the nodes of the new list

are in sorted order wrt index. The size of head node of new list becomes the larger of two sizes of list1 and list2. Example of add and merge is shown in at one of the page of your answer sheet. [1+]

Example: List 1:



List 2:



After add and merger operation, new list becomes as follows:

